

Lecture 14

Network Security & Cryptography

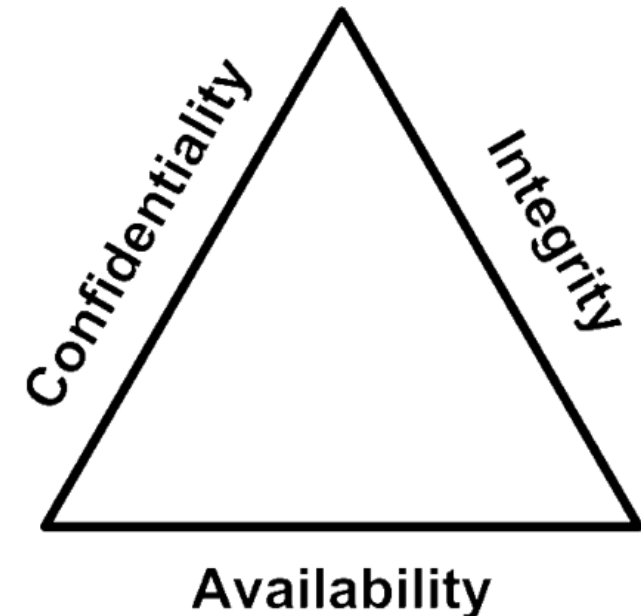
CSC175

Outline

- Overview
- Symmetric encryption
- Public-key cryptography
- Message authentication and hash functions
- Digital signatures
- Random and pseudorandom numbers

The CIA Triad

- Confidentiality: only sender, intended receiver should “understand” message contents
 - sender encrypts message
 - receiver decrypts message
- Integrity: sender, receiver want to ensure message not altered (in transit, or afterwards) without detection
- Availability: services must be accessible and available to users



CIA Examples

	Availability	Confidentiality	Integrity
Hardware	Equipment is stolen or disabled, thus denying service.	An unencrypted CD-ROM or DVD is stolen.	
Software	Programs are deleted, denying access to users.	An unauthorized copy of software is made.	A working program is modified, either to cause it to fail during execution or to cause it to do some unintended task.
Data	Files are deleted, denying access to users.	An unauthorized read of data is performed. An analysis of statistical data reveals underlying data.	Existing files are modified or new files are fabricated.
Communication Lines and Networks	Messages are destroyed or deleted. Communication lines or networks are rendered unavailable.	Messages are read. The traffic pattern of messages is observed.	Messages are modified, delayed, reordered, or duplicated. False messages are fabricated.

Vulnerabilities, Threats and Attacks

- Categories of vulnerabilities
 - Corrupted (loss of integrity)
 - Leaky (loss of confidentiality)
 - Unavailable or very slow (loss of availability)
- Threats
 - Capable of exploiting vulnerabilities
 - Represent potential security harm to an asset
- Attacks (threats carried out)
 - Insider vs. outsider
 - Passive vs. active

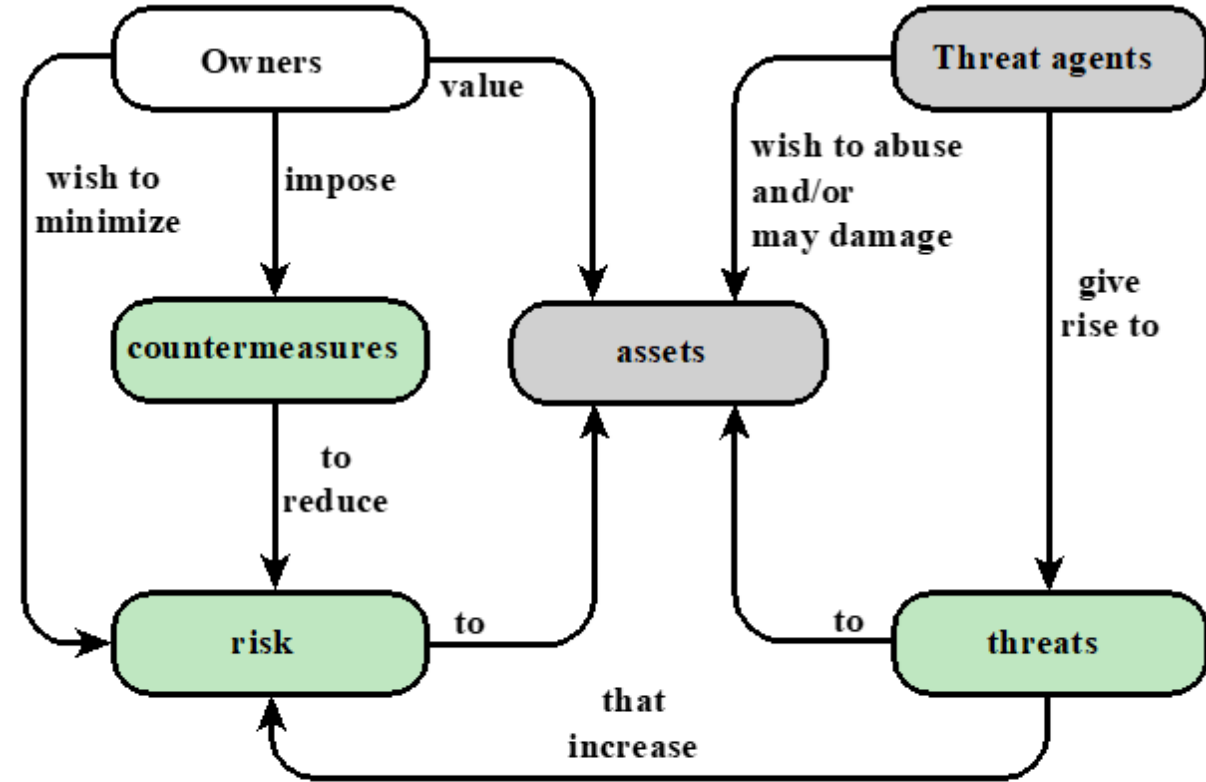


Figure 1.2 Security Concepts and Relationships

Passive Attacks to Compromise Confidentiality

- In passive attacks, attacker attempts to learn or make use of information from the system but does not affect system resources
 - Eavesdropping on, or monitoring of, transmissions
 - Goal of attacker is to obtain information that is being transmitted
- Two categories :
 - Leaking of message content
 - Traffic analysis: attacker infers information from network traffic patterns, even though message content is not leaked (e.g., with encryption)

Active Attacks to Compromise Integrity & Availability

- In active attacks, attacker attempts to alter system resources or affect their operation.
 - Involve some modification of the data stream or the creation of a false stream.
- Four categories:
 - Replay: attacker captures a message and subsequent retransmits it to produce an unauthorized effect.
 - Masquerade / Impersonation: one entity pretends to be a different entity, e.g., can fake (spoof) source address in packet (or any field in packet)
 - Modification of messages: some portion of a message is altered, or messages are delayed or reordered.
 - Denial of Service: prevents or inhibits the normal use of the target system.

Countermeasures

- Means used to deal with security attacks
 - Prevent, Detect, Recover
 - Goal is to minimize residual level of risk to the assets

Attack Surface

- An attack surface consists of the reachable and exploitable vulnerabilities in a system, including:
 - Network Attack Surface
 - Vulnerabilities over an enterprise network, wide-area network, or the Internet
 - Including network protocol vulnerabilities, such as those used for DoS attacks
 - Software Attack Surface
 - Vulnerabilities in application, utility, or operating system code
 - Human Attack Surface
 - Social engineering, human error, and trusted insiders

Security Risk

- Defense in depth is a concept used in information security in which multiple layers of security controls (defense) are placed throughout the system.
- Security risk can be determined by defense in depth layering (shallow or deep) and attack Surface (small or large).

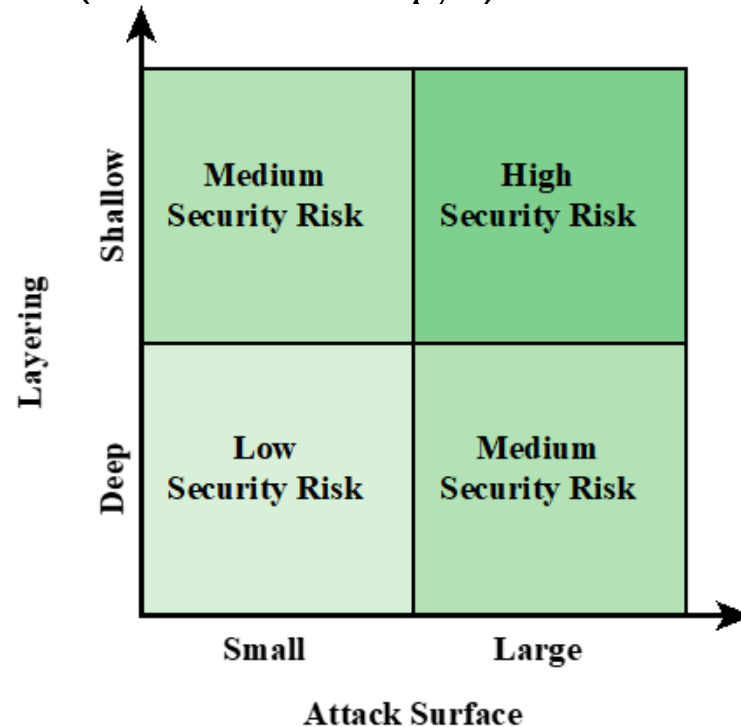


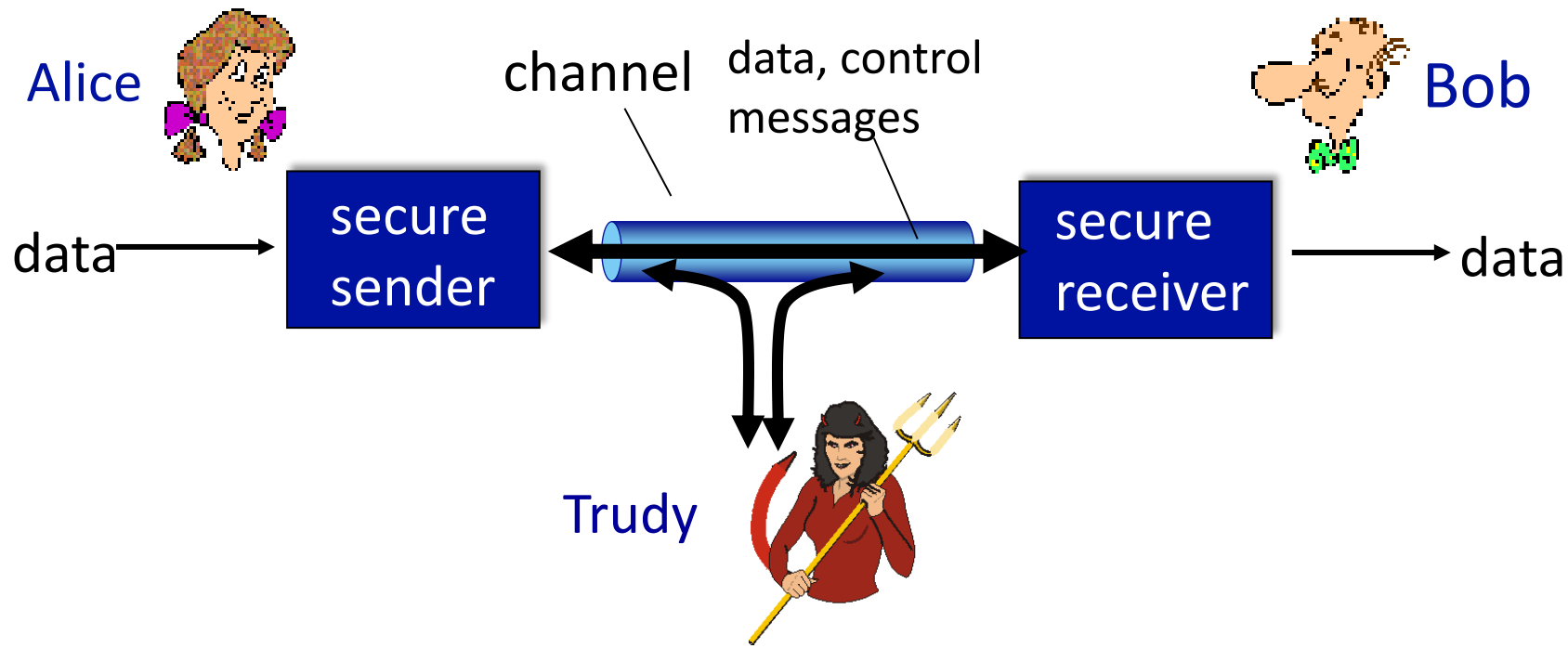
Figure 1.4 Defense in Depth and Attack Surface

Terminology

- Plaintext
 - Also called cleartext
- Ciphertext
 - Scrambled message produced as output
- Encryption algorithm
 - Transforms plaintext to ciphertext
- Decryption algorithm
 - Transforms ciphertext to plaintext

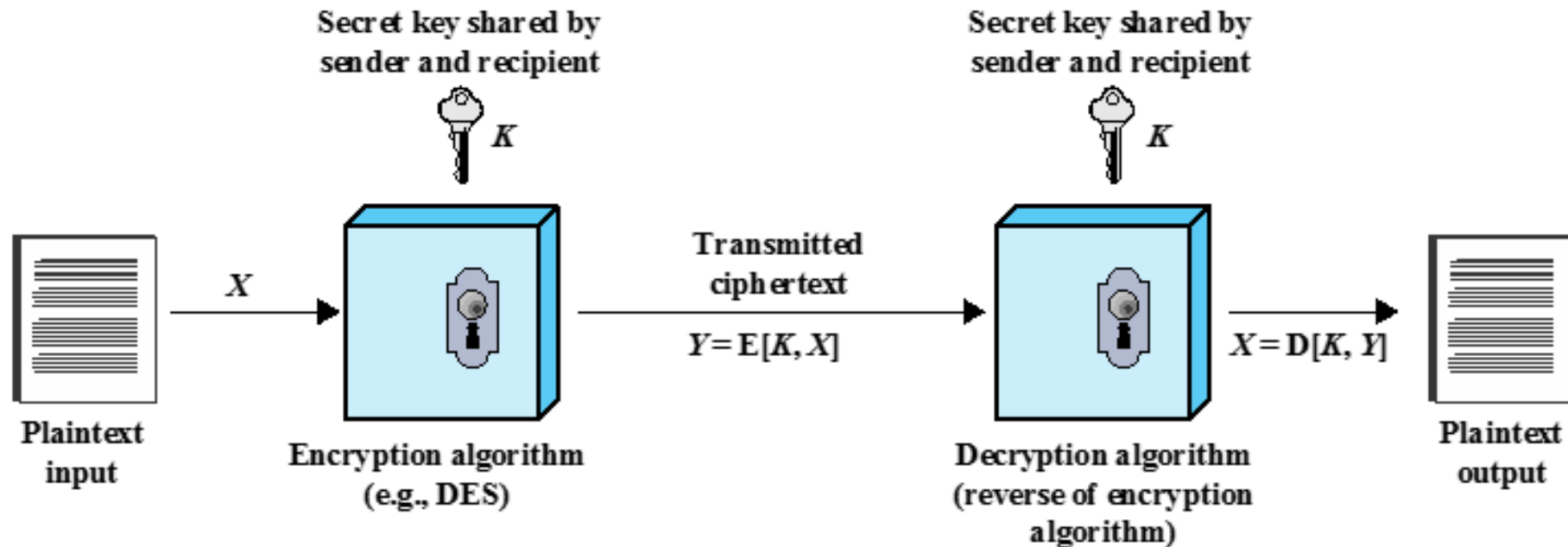
Friends and enemies: Alice, Bob, Trudy

- Well-known in network security world
- Bob, Alice want to communicate “securely”
- Trudy (intruder) may intercept, delete, add messages



Symmetric Encryption

- $\text{Encrypted_Message} = \text{Encrypt}(\text{Key}, \text{Message})$
- $\text{Message} = \text{Decrypt}(\text{Key}, \text{Encrypted_Message})$
- Also called secret-key cryptography, for protecting confidentiality
 - Sender and receiver must share the same secret key



Simple encryption scheme

substitution cipher: substituting one thing for another

- monoalphabetic cipher: substitute one letter for another

plaintext:	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
		↓																								↓
ciphertext:	m	n	b	v	c	x	z	a	s	d	f	g	h	j	k	l	p	o	i	u	y	t	r	e	w	q

e.g.: Plaintext: bob. i love you. alice
ciphertext: nkn. s gktc wky. mgsbc

🔑 *Encryption key*: mapping from set of 26 letters
to set of 26 letters

A more sophisticated encryption approach

- n substitution ciphers, $M_1, M_2, \dots, M_n$
 - cycling pattern:
 - e.g., $n=4$: M_1, M_3, M_4, M_3, M_2 ; M_1, M_3, M_4, M_3, M_2 ; ..
 - for each new plaintext symbol, use subsequent substitution pattern in cyclic pattern
 - dog: d from M_1 , o from M_3 , g from M_4
- 🔑 *Encryption key*: n substitution ciphers, and cyclic pattern

Block & Stream Ciphers

■ Stream Cipher

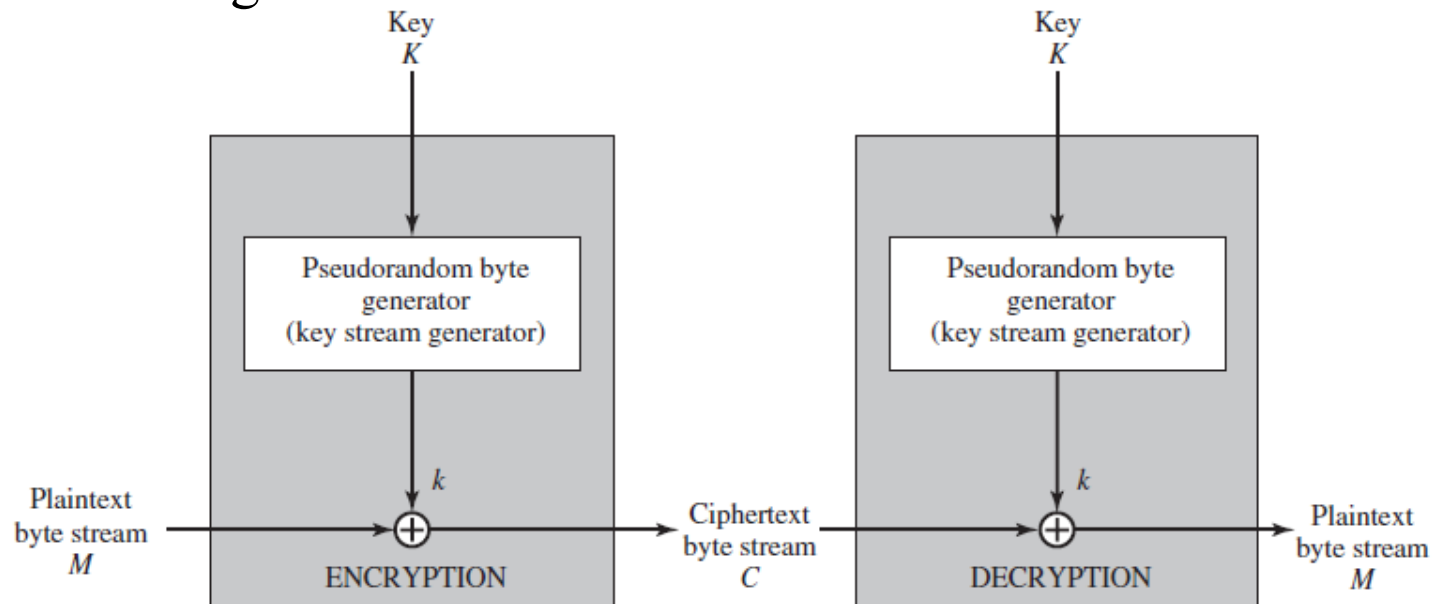
- Processes the input elements continuously, producing output one element at a time
- One element may be 1 bit, 1 Byte, or more than 1 Byte
- Faster than block ciphers

■ Block Cipher

- Processes input data one block at a time
- Produces an output block for each input block
- We focus on block cipher in this lecture

An Example Stream Cypher

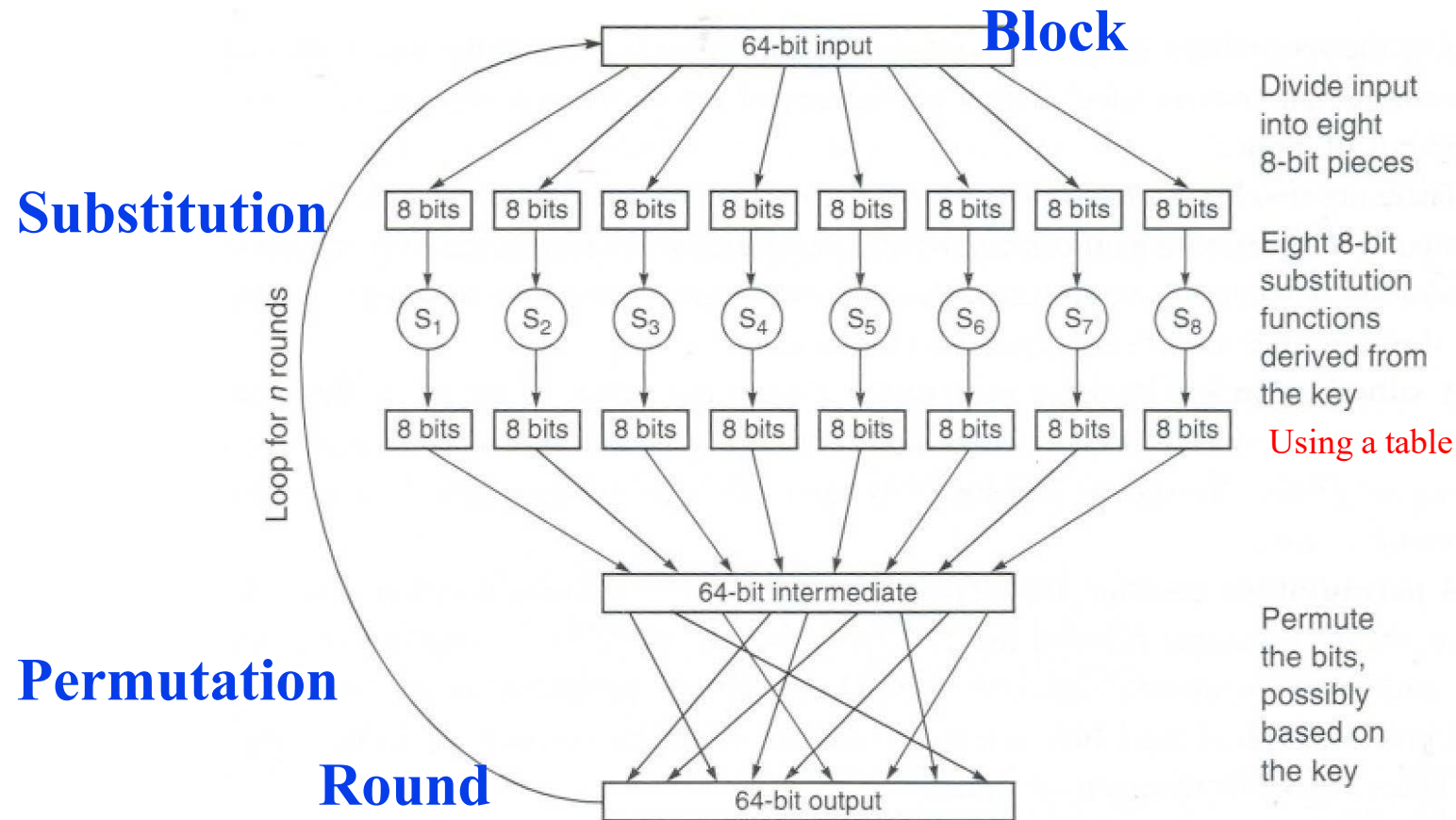
- A stream cypher that stream cipher that operates one bit at a time:
 - Sender and receiver share a secrete key K , which can be input to a pseudorandom byte generator that produces a pseudorandom stream of bytes, called a keystream $KS(k$ in the figure).
 - The plaintext is XOR'ed with KS bit-by-bit to produce the cyphertext: $C_i = M_i \text{ XOR } KS_i$
 - The cyphertext is XOR'ed with the same keystream KS to recover the plaintext $M_i = C_i \text{ XOR } KS_i$
 - This relies on sender and receiver sharing a secrete key K and using the same key stream generator algorithm



(b) Stream encryption

Block Encryption

- Block Encryption for each block



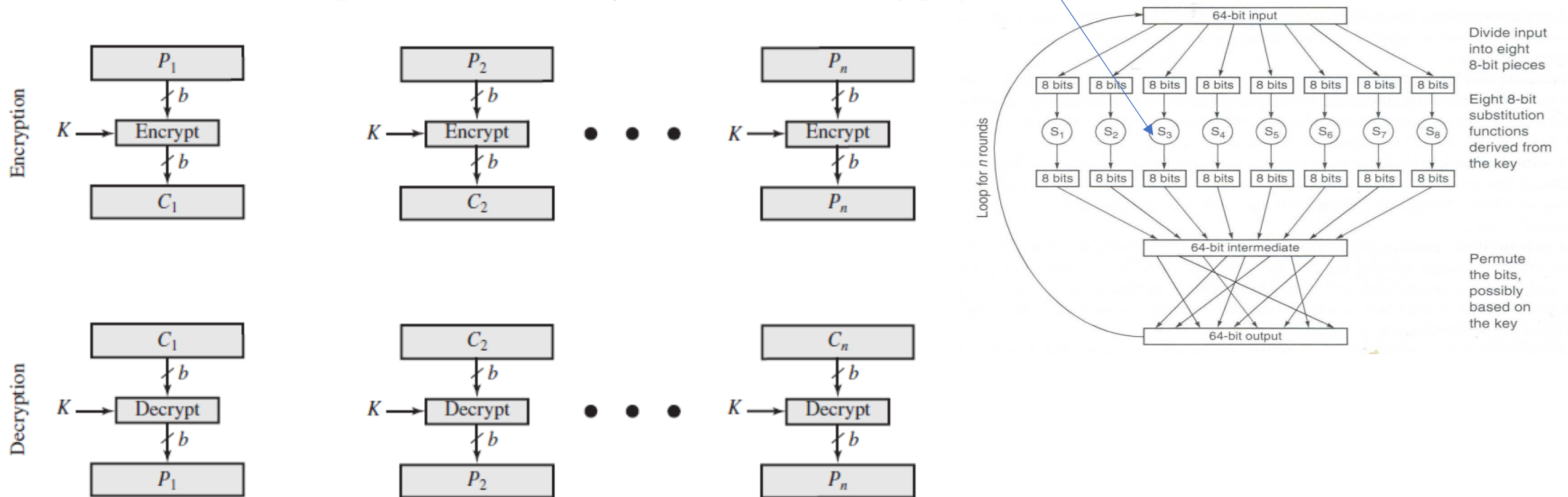
Block Encryption (Cont)

- ❑ 64-bit block
 - ❑ Short block length $\Rightarrow$ tabular attack
- ❑ Transformations:
 - Substitution: replace k-bit input blocks with k-bit output blocks
 - Permutation: move input bits around. $1 \rightarrow 13, 2 \rightarrow 61$, etc.
- ❑ Round: Substitution round followed by permutation round to achieve Diffusion + Confusion.

Diffusion $\Rightarrow$ 1-bit change in input changes many bits in the output.
Confusion $\Rightarrow$ Relationship between input and output is complex.

Block Cipher: Electronic CodeBook (ECB)

- Each plaintext block P_i (e.g., 64 or 128 bits) is encoded independently using the same key K
 - Attacker may exploit regularities in the plaintext to perform cryptanalysis, since same plaintext block generates same cyphertext block.



(a) Block cipher encryption (electronic codebook mode)

Block Cipher: Cipher Block Chaining (CBC)

- Goal: The same block is encoded differently each time.
- Input to the encryption algorithm is the XOR of the plaintext block P_i and the preceding ciphertext block C_{i-1} . The first block is XORed with an Initialization Vector (IV),
 - P_i and C_{i-1} have no fixed relationship, removing the regularities in the plaintext for ECB, hence more secure.
- Exclusive OR (XOR) operator
 - $0 \text{ XOR } 0 = 0$
 - $0 \text{ XOR } 1 = 1$
 - $1 \text{ XOR } 0 = 1$
 - $1 \text{ XOR } 1 = 0$
 - $0101 \text{ XOR } 0011 = 0110$
 - Properties
 - $C_{j-1} \text{ XOR } C_{j-1} = 0$ (any string XOR'ed with itself is all 0's).
 - $0 \text{ XOR } P_j = P_j$ (0 XOR'ed with any string is the same string).

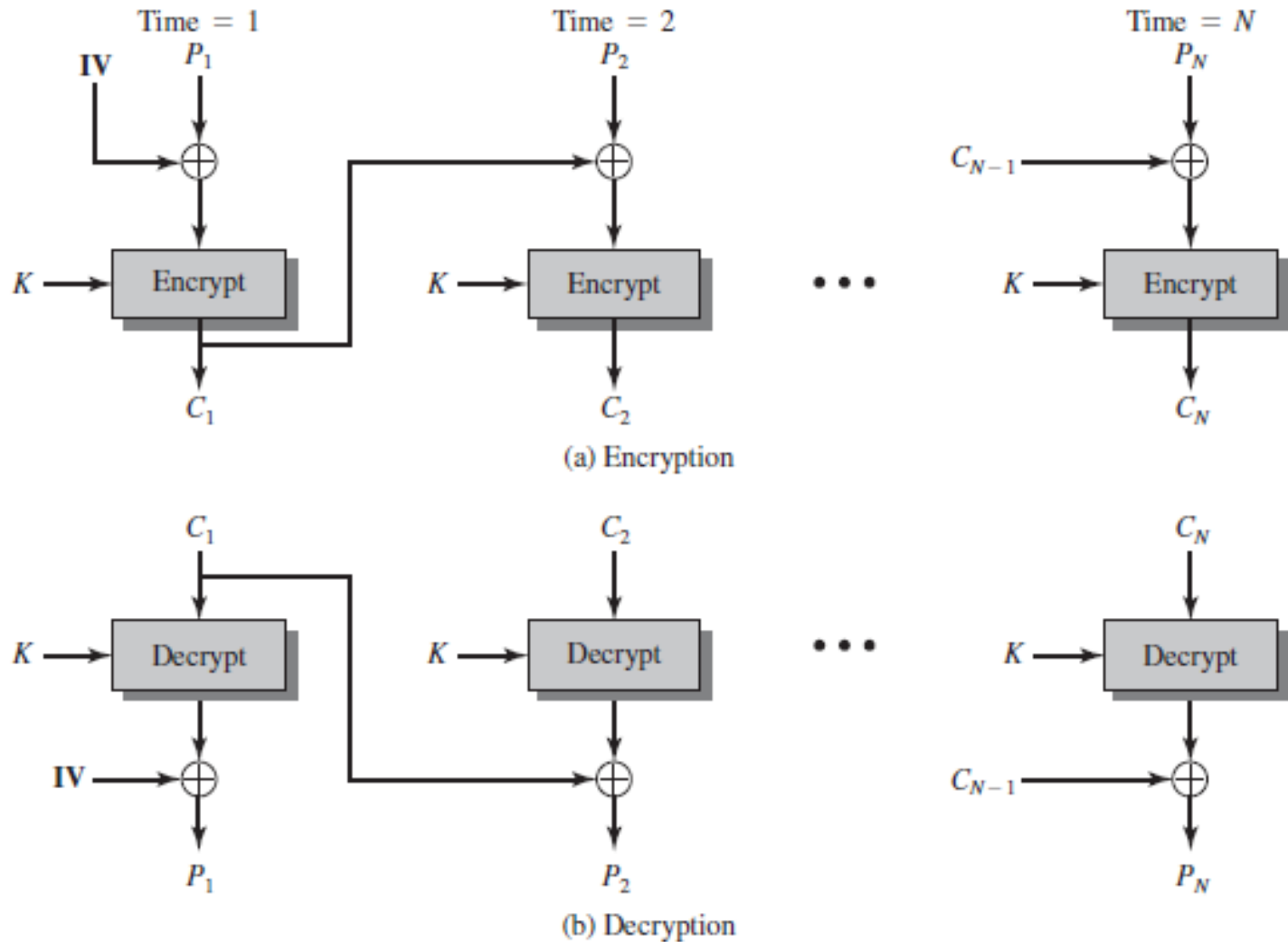


Figure 20.7 Cipher Block Chaining (CBC) Mode

Data Encryption Standard (DES)

- Published by National Institute of Standards and Technology (NIST) in 1977
- For commercial and unclassified government applications
- Eight-octet (64-bit) key.
- Efficient hardware implementation
- Used in most financial transactions
- Moore's Law → computing power doubles → brute force becomes feasible
- 56-bit was secure in 1977 but is not secure today

Triple DES (3DES)

- Repeats DES three times using either 2 or 3 unique keys, with total key length of 112 or 168 bits
 - Ciphertext = $\text{DES}(K_1, \text{DES}(K_2, \text{DES}(K_1, \text{Plain Text})))$ or
 - Ciphertext = $\text{DES}(K_3, \text{DES}(K_2, \text{DES}(K_1, \text{Plain Text})))$
- Pros:
 - Key length of 112/168 bits removes the vulnerability to brute-force attacks
- Cons:
 - Performance is slow: three times as many calculations as DES.
 - Block size larger than 64 bits is desirable for efficiency and security.

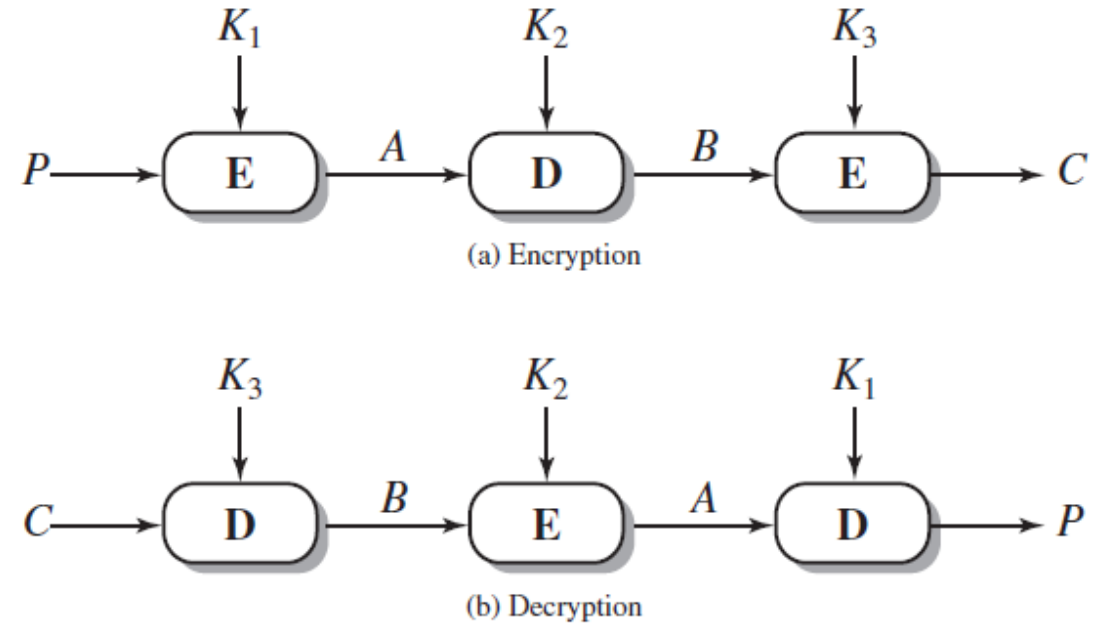


Figure 20.2 Triple DES

Advanced Encryption Standard (AES)

- ❑ Designed in 1997-2001 by National Institute of Standards and Technology (NIST) as Federal information processing standard (FIPS 197)
- ❑ A symmetric block cipher with a block length of 128 bits
- ❑ Key lengths 128, 192, and 256 bits.

	DES	Triple DES	AES
Plaintext block size (bits)	64	64	128
Ciphertext block size (bits)	64	64	128
Key size (bits)	56	112 or 168	128, 192, or 256

DES = Data Encryption Standard

AES = Advanced Encryption Standard

Attacks against Symmetric Encryption

- **Cipher-text only attack:**
Trudy has ciphertext she can analyze
- **Two approaches:**
 - brute force: search through all keys
 - statistical analysis
- **Known-plaintext attack:**
Trudy has plaintext corresponding to ciphertext
 - *e.g.*, in monoalphabetic cipher, Trudy determines pairings for a,l,i,c,e,b,o,
- **Chosen-plaintext attack:**
Trudy can get ciphertext for chosen plaintext

Time Required for Brute-Force Attack

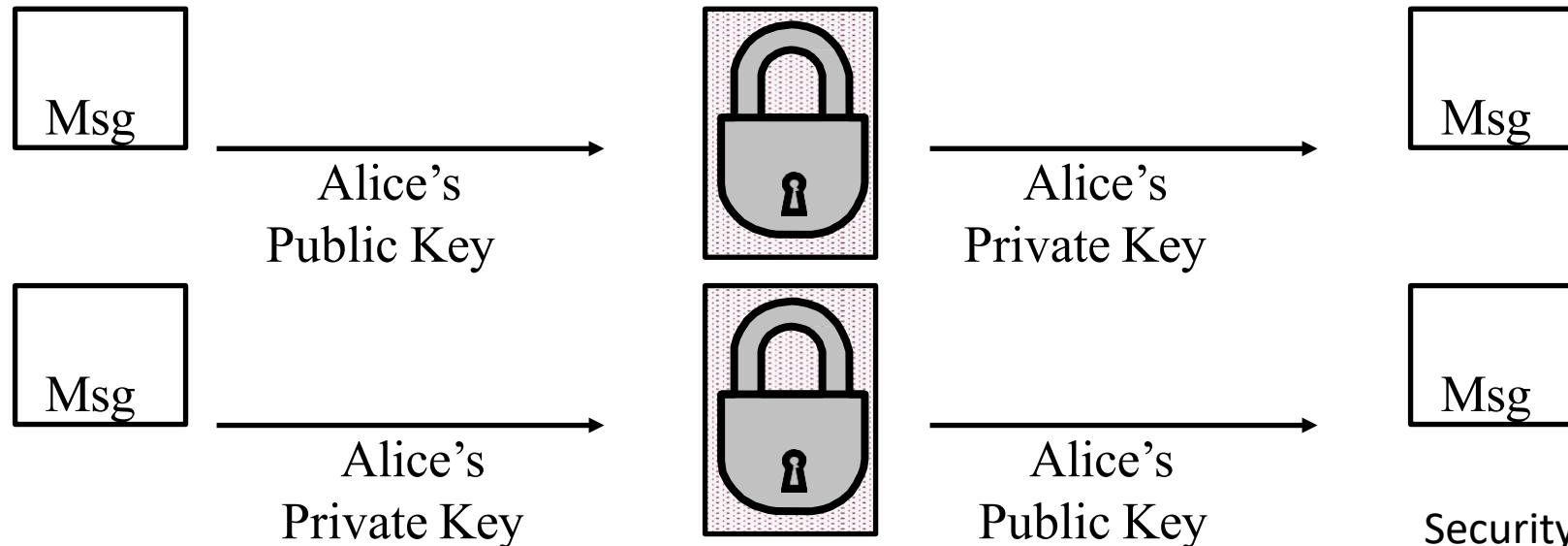
Key size (bits)	Cipher	Number of Alternative Keys	Time Required at 10^9 decryptions/s	Time Required at 10^{13} decryptions/s
56	DES	$2^{56} \approx 7.2 \times 10^{16}$	2^{55} ns = 1.125 years	1 hour
128	AES	$2^{128} \approx 3.4 \times 10^{38}$	2^{127} ns = 5.3×10^{21} years	5.3×10^{17} years
168	Triple DES	$2^{168} \approx 3.7 \times 10^{50}$	2^{167} ns = 5.8×10^{33} years	5.8×10^{29} years
192	AES	$2^{192} \approx 6.3 \times 10^{57}$	2^{191} ns = 9.8×10^{40} years	9.8×10^{36} years
256	AES	$2^{256} \approx 1.2 \times 10^{77}$	2^{255} ns = 1.8×10^{60} years	1.8×10^{56} years

Symmetric Key Encryption: Summary

1. Secret key encryption requires a shared secret key
2. Block encryption, e.g., DES, 3DES, AES, break cleartext into fixed-size blocks and encrypt
3. CBC ensures that the same plain text results in different ciphertexts.

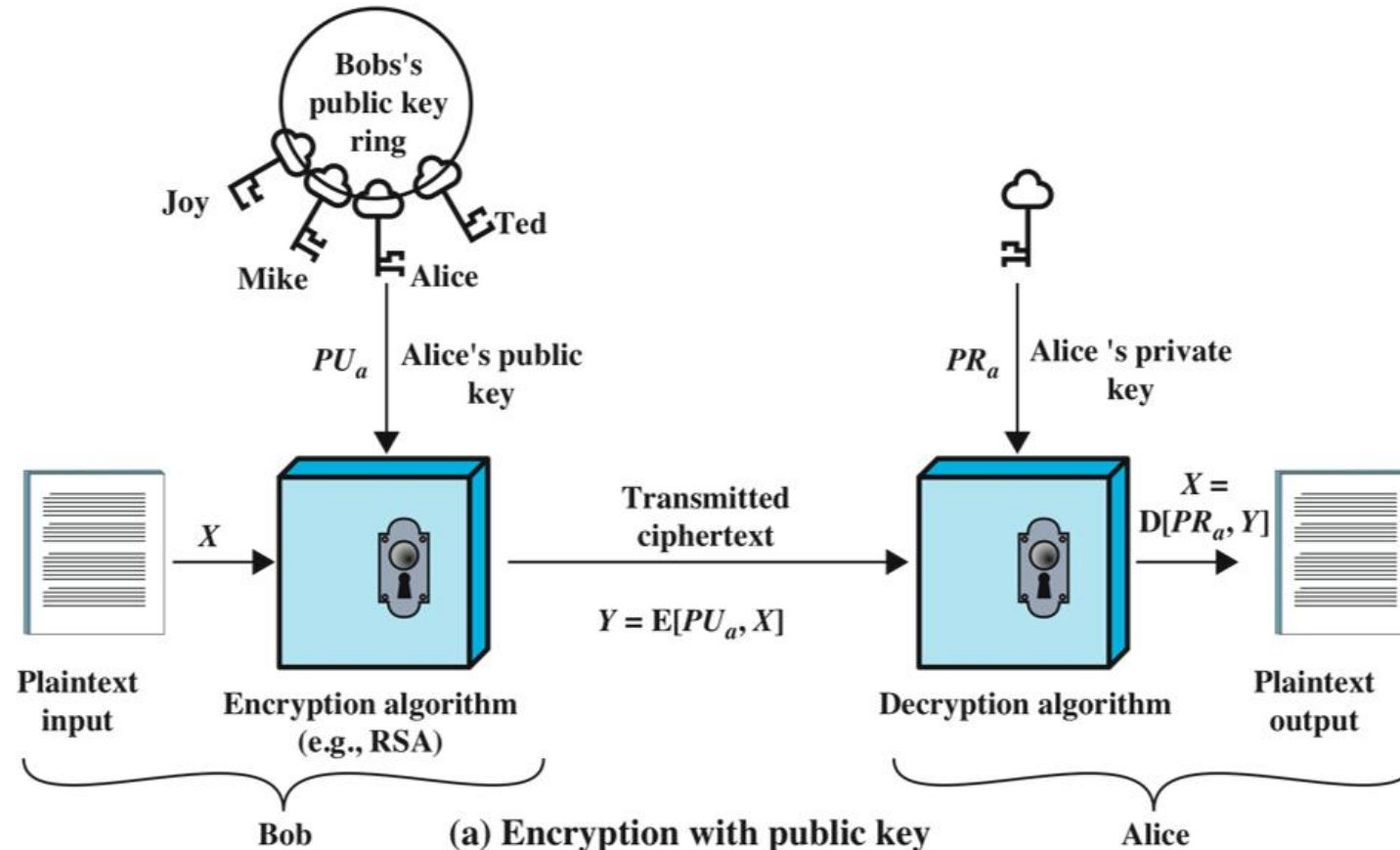
Public Key Cryptography

- ❑ Each user has two separate keys
 - ❑ PU_a : Alice's Public Key; PR_a : Alice's Private Key
 - ❑ PU_b : Bob's Public Key; PR_b : Bob's Private Key
- ❑ Encrypted with private key, decrypted with public key
 - ❑ $Message = Decrypt(PU_a, Encrypt(PR_a, Message))$
- ❑ Encrypted with public key, decrypted with private key
 - ❑ $Message = Decrypt(PR_a, Encrypt(PU_a, Message))$
- ❑ Requirement: given public key, it is computationally infeasible to compute private key



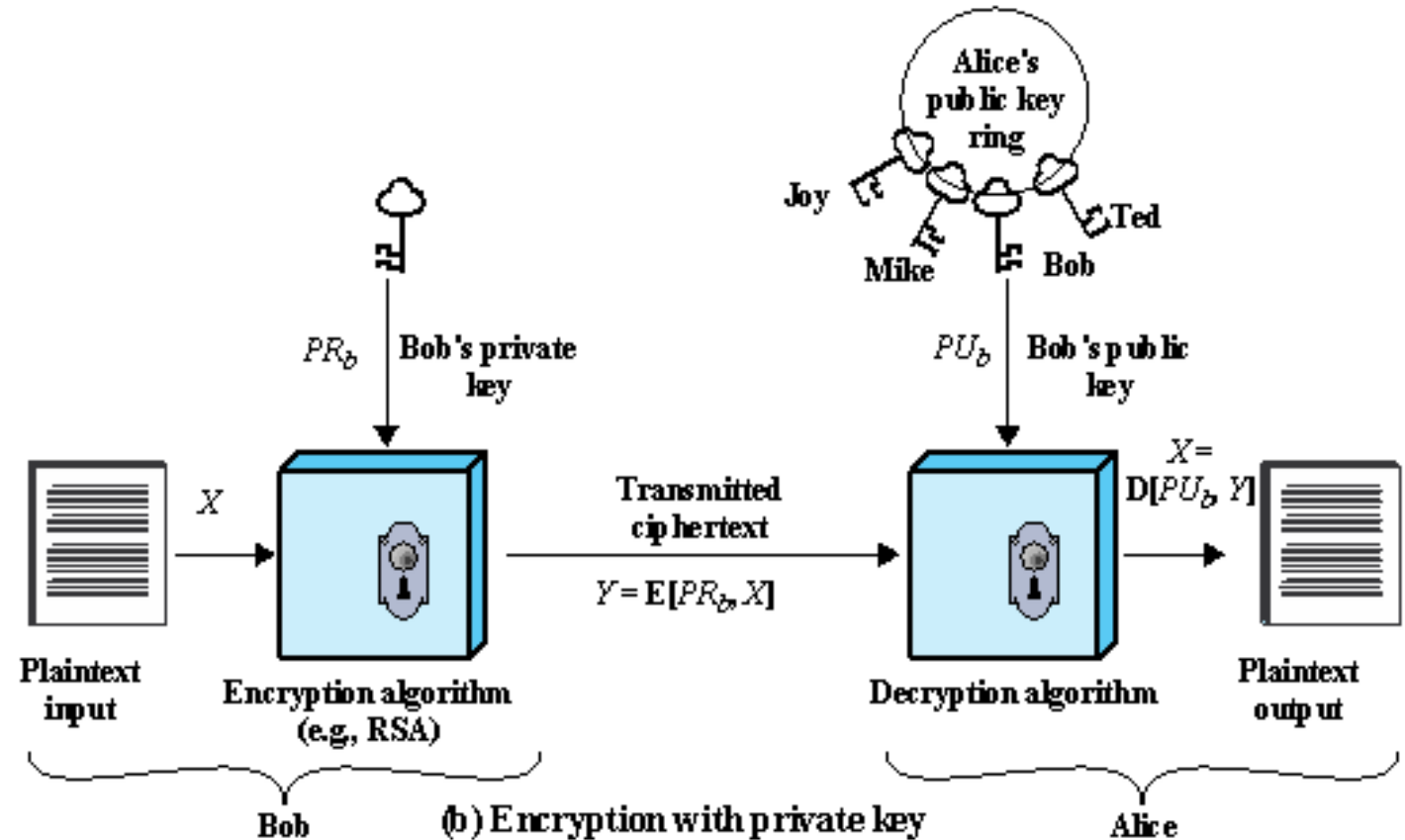
Public-Key Crypto for Confidentiality

- Sender encrypts data using the receiver's public key
- Receiver decrypts data using his own private key
 - Use of a public/private key pair removes the need for sharing a secret key



Public-Key Crypto for Integrity and Non-Repudiation

- Sender encrypts data using his or her private key
- Receiver, or anyone else, can decrypt the message using sender's public key.
 - There are more efficient methods based on MAC or crypto hash function (discussed later).



Public-Key Crypto for both Confidentiality and Integrity/Non-Repudiation

- ❑ Combine the previous two approaches
- ❑ Bob sends message to Alice
- ❑ Bob performs: $\text{Encrypted_Message} = \text{Encrypt}(\text{PU}_a \text{ ublic_Key_Alice}, \text{Encrypt}(\text{PR}_b, \text{Message}))$
- ❑ Alice performs: $\text{Cleartext_Message} = \text{Decrypt}(\text{Public_Key_Bob}, \text{Decrypt}(\text{Private_Key_Alice}, \text{Encrypted_Message}))$



Public-Key Crypto Algorithms and Protocols

- RSA (Rivest-Shamir-Adelman)
 - Key generation, encryption, decryption
- Elliptic Curve Cryptography (ECC)
 - Lightweight public key algorithm for embedded and IoT devices
- Diffie-Hellman key exchange protocol
 - Used to establish a shared secret between two parties, e.g., a secret key for symmetric encryption
 - Diffie-Hellman is for key exchange, not encryption itself with the keys

Prerequisite: Modular Arithmetic

- $x \bmod n$ = remainder of x when divide by n

- facts:

$$[(a \bmod n) + (b \bmod n)] \bmod n = (a+b) \bmod n$$

$$[(a \bmod n) - (b \bmod n)] \bmod n = (a-b) \bmod n$$

$$[(a \bmod n) * (b \bmod n)] \bmod n = (a*b) \bmod n$$

- thus

$$(a \bmod n)^d \bmod n = a^d \bmod n$$

- example: $x=14$, $n=10$, $d=2$:

$$(x \bmod n)^d \bmod n = 4^2 \bmod 10 = 6$$

$$x^d = 14^2 = 196 \quad x^d \bmod 10 = 6$$

Diffie-Hellman

- Alice and Bob agree on:
 - A sufficiently large prime number p
 - A generator number $g < p$
- Alice chooses a secret number a and sends to Bob
 - $A = g^a \bmod p$ (One-way function, easy to compute A from g, p , but hard to obtain g, p from A .)
- Bob chooses a secret number b and sends to Alice
 - $B = g^b \bmod p$
- Alice computes $S_1 = B^a \bmod p$
- Bob also computes $S_2 = A^b \bmod p$
- We can easily show that $S_1 = S_2$, and this is their shared secret key
 - $S_1 = B^a \bmod p = g^{ba} \bmod p = g^{ab} \bmod p = A^b \bmod p = S_2$
- p, g, A, B are all public; only Alice knows secret a ; only Bob knows secret b . At the end, Alice and Bob have a shared secret $S_1 = S_2$.

Diffie-Hellman Example

- Alice and Bob agree on:
 - A sufficiently large prime number $p = 17$
 - A generator number $g = 3 < 17$
- Alice chooses a secret number $a = 15$ and sends to Bob
 - $A = 3^{15} \bmod 17 = 6$
- Bob chooses a secret number $b = 13$ and sends to Alice
 - $B = 3^{13} \bmod 17 = 12$
- Alice computes $S_1 = 12^{15} \bmod 17 = 10$
- Bob also computes $S_2 = 6^{13} \bmod 17 = 10$
- We can easily show that $S_1 = S_2$, and this is their shared secret key
 - $S_1 = 12^{15} \bmod 17 = 3^{13 \cdot 15} \bmod p = 3^{15 \cdot 13} \bmod 17 = 6^{13} \bmod 17 = S_2 = 10$
- p, g, A, B are all public; only Alice knows secret $a = 15$; only Bob knows secret $b = 13$. At the end, Alice and Bob have a shared secret $S_1 = S_2 = 10$.

Requirements for Public-Key Crypto

- Computationally easy to
 - create key pairs
 - encrypt/decrypt messages using either public or private key
- Computationally infeasible to
 - determine private key from public key
 - recover cleartext without key
- Either key can be used for each role (public/private key)

Public-Key Crypto in Practice

- Public-key crypto, e.g., RSA is computationally intensive
 - DES is at least 100 times faster than RSA
- In practice: use public key crypto to establish secure connection, then use symmetric encryption for encrypting data

session key, K_s

- Bob and Alice use public-key crypto to exchange a symmetric session key K_s
- Then use K_s for symmetric encryption/decryption

Public Key Crypto: Summary

Public Key Encryption uses two keys: Public and Private.

1. Public key → used for encryption (confidentiality)
2. Private key → used for decryption

Or

1. Private key → used for signing
2. Public key → used for verification

Random Numbers

- Used to generate:
 - Keys for public-key algorithms
 - Stream key for symmetric stream cipher
 - Symmetric key for use as a temporary session key or in creating a digital envelope
 - Handshaking to prevent replay attacks
 - Session key

Random Number Requirements

- Randomness

- Uniform distribution
 - Frequency of occurrence of each of the numbers should be approximately the same
- Independence
 - No one value in the sequence can be inferred from the others

- Unpredictability

- Each number is statistically independent of others in the sequence, so future elements of the sequence cannot be predicted based on past elements

Pseudorandom vs. Random Numbers

- Pseudorandom numbers are generated with deterministic algorithms with random seeds:
 - The same seed results in the same sequence of random numbers
 - The sequence produced are not truly statistically random, but may pass many reasonable tests of randomness
- True random number generator (TRNG):
 - Uses a nondeterministic source to produce randomness
 - Most operate by measuring unpredictable natural processes
 - e.g. radiation, gas discharge, leaky capacitors